

一、整合概述

ph_mqtt 是 Eclipse Paho MQTT C Client 的 OpenHarmony 移植版本，实现了 MQTT 3.1.1 和 MQTT 5.0 协议。本文档记录了 ph_mqtt 在 OpenHarmony hispark_hi3519dv500_linux 产品中的整合过程。

版本信息：

- ph_mqtt 版本：3.1
- OpenHarmony 版本：4.0-Release
- 目标产品：hispark_hi3519dv500_linux
- 目标架构：ARM64 (aarch64)
- 编译日期：2026-05-18

适配模式： 模式 D（无参考仓库适配）——通过注册组件 + 修复已知 ARM64/musl 编译问题完成适配。

依赖关系：

```
ph_mqtt (libpaho-mqtt3as.so)
├─ libssl_openssl.so - OpenSSL SSL/TLS 库
├─ libcrypto_openssl.so - OpenSSL 加密库
└─ libc.so (musl) - C 标准库
```

二、整合步骤

1. 产品配置修改

文件： vendor/hisilicon/hispark_hi3519dv500_linux/config.json

在 `thirdparty` 子系统中添加 ph_mqtt 组件：

修改前：

```
{
  "subsystem": "thirdparty",
  "components": [
    ...
    { "component": "tcpdump", "features": [] }
  ]
}
```

修改后：

```
{
  "subsystem": "thirdparty",
  "components": [
    ...
    { "component": "tcpdump", "features": [] },
    { "component": "ph_mqtt", "features": [] }
  ]
}
```

2. 构建脚本修改

文件: `third_party/ph_mqtt/BUILD.gn`

由于 ARM64 musl libc 头文件兼容性问题, 需要从 `cflags` 中移除 `-D_UAPI_LINUX_SOCKET_H`。该修改涉及 `lite` 分支 (原第 63 行) 和标准系统分支 (原第 97 行) 两处。

lite 分支修改前:

```
cflags = [  
    "-w",  
    "-fPIC",  
    "-Os",  
    "-fvisibility=hidden",  
    "-DOPENSSL",  
    "-D_GNU_SOURCE",  
    "-DPAHO_MQTT_EXPORTS=1",  
    "-DOPENSSL_ARM_PLATFORM",  
    "-D_UAPI_LINUX_SOCKET_H",  
    "-D__kernel_sa_family_t=sa_family_t",  
    "-DSSL_get_peer_certificate=SSL_get1_peer_certificate"  
]
```

lite 分支修改后:

```
cflags = [  
    "-w",  
    "-fPIC",  
    "-Os",  
    "-fvisibility=hidden",  
    "-DOPENSSL",  
    "-D_GNU_SOURCE",  
    "-DPAHO_MQTT_EXPORTS=1",  
    "-DOPENSSL_ARM_PLATFORM",  
    "-D__kernel_sa_family_t=sa_family_t",  
    "-DSSL_get_peer_certificate=SSL_get1_peer_certificate"  
]
```

标准系统分支做相同修改, 同样移除 `-D_UAPI_LINUX_SOCKET_H`, 保留 `-D__kernel_sa_family_t=sa_family_t`。

修改原因: ARM64 的 musl sysroot 中, `struct sockaddr_storage` 的定义被 `#ifndef __MUSL__` 守卫跳过, 而 `-D_UAPI_LINUX_SOCKET_H` 又阻止了内核 `linux/socket.h` 头文件的包含, 导致 `sockaddr_storage` 类型完全缺失。移除该宏后, 由内核头文件正常提供 `sockaddr_storage` 定义, `-D__kernel_sa_family_t=sa_family_t` 避免类型冲突。

三、问题解决记录

问题 1: sockaddr_storage 类型未定义

错误信息:

```
sysroot/usr/include/netinet/in.h:281:26: error: field 'gr_group' has incomplete type
281 | struct sockaddr_storage gr_group;
```

根因分析:

ARM64 平台的 musl sysroot 头文件链路:

1. `sys/socket.h` 中 `sockaddr_storage` 被 `#ifndef __MUSL__` 守卫——因为 `__MUSL__` 已定义, musl 不提供该类型定义
2. 正常情况下由 `linux/socket.h` (内核头文件) 提供 `sockaddr_storage` 定义
3. BUILD.gn 中 `-D_UAPI_LINUX_SOCKET_H` 宏导致 `linux/socket.h` 整个被跳过 (`#ifndef _UAPI_LINUX_SOCKET_H` 守卫)
4. 结果: `sockaddr_storage` 两边都没定义, `netinet/in.h` 中 `struct group_req` 引用该类型时报错

include 链路:

```
Socket.h:50 → <sys/socket.h>
Socket.h:60 → <netinet/in.h>
netinet/in.h:281 → struct sockaddr_storage gr_group (未定义!)
```

解决方案: 移除 `-D_UAPI_LINUX_SOCKET_H`, 让内核头文件正常包含, 保留 `-D__kernel_sa_family_t=sa_family_t` 处理类型兼容性。

与 taurus 平台差异: taurus 使用 ARM 32-bit 工具链, 其 musl sysroot 的 `sys/socket.h` 可能未用 `#ifndef __MUSL__` 保护 `sockaddr_storage`, 因此 `sys/socket.h` 自身就能提供定义, `-D_UAPI_LINUX_SOCKET_H` 不影响编译。

四、编译验证

编译命令

```
# 单独编译 ph_mqtt
hb build -T libpaho-mqtt3as
```

编译结果

```
[OHOS INFO] [1045/1045] SOLINK ./libpaho-mqtt3as.so
[OHOS INFO] ipcamera_hispark_hi3519dv500_linux build success
[OHOS INFO] Cost time: 0:02:55
```

生成文件

```
# 共享库 (stripped)
out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/libpaho-mqtt3as.so

# 共享库 (unstripped, 包含调试符号)
out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/unstripped/usr/lib/libpaho-mqtt3as.so
```

文件信息

```
$ ls -lh out/.../libpaho-mqtt3as.so
-rwxrwxr-x 1 chenzhy223 chenzhy223 207K 5月 18 20:39 libpaho-mqtt3as.so

$ file out/.../libpaho-mqtt3as.so
libpaho-mqtt3as.so: ELF 64-bit LSB shared object, ARM aarch64, version 1 (SYSV),
dynamically linked, stripped
```

文件特性:

- 文件大小: 207KB (stripped) / 243KB (unstripped)
- 架构: ARM 64-bit (aarch64)
- 链接方式: 动态链接
- 已 stripped

依赖库检查

```
$ readelf -d out/.../libpaho-mqtt3as.so | grep NEEDED
0x0000000000000001 (NEEDED) Shared library: [libssl_openssl.so]
0x0000000000000001 (NEEDED) Shared library: [libcrypto_openssl.so]
0x0000000000000001 (NEEDED) Shared library: [libc.so]
```

导出符号验证

```
$ nm -D out/.../unstripped/usr/lib/libpaho-mqtt3as.so | grep " T MQTTAsync"
0000000000009098 T MQTTAsync_connect
000000000000bee0 T MQTTAsync_create
000000000000b920 T MQTTAsync_createWithOptions
0000000000008e64 T MQTTAsync_destroy
000000000000a8bc T MQTTAsync_disconnect
0000000000008e04 T MQTTAsync_global_init
000000000000a984 T MQTTAsync_isConnected
0000000000009ba4 T MQTTAsync_reconnect
000000000000a460 T MQTTAsync_send
000000000000a528 T MQTTAsync_sendMessage
000000000000ab28 T MQTTAsync_subscribe
000000000000ac24 T MQTTAsync_unsubscribe
000000000000aa18 T MQTTAsync_setCallbacks
000000000000c1f0 T MQTTAsync_free
```

```
000000000000c128 T MQTTAsync_freeMessage
000000000000b7f8 T MQTTAsync_getVersionInfo
```

```
// 创建客户端
int MQTTAsync_create(MQTTAsync* handle, const char* serverURI,
                    const char* clientId, int persistence_type,
                    void* persistence_context);

// 销毁客户端
void MQTTAsync_destroy(MQTTAsync* handle);

// 获取版本信息
MQTTAsync_nameValue* MQTTAsync_getVersionInfo(void);
```

```
// 连接到服务器
int MQTTAsync_connect(MQTTAsync handle, const MQTTAsync_connectOptions* options);

// 断开连接
int MQTTAsync_disconnect(MQTTAsync handle, const MQTTAsync_disconnectOptions* options);

// 检查连接状态
int MQTTAsync_isConnected(MQTTAsync handle);
```

[illegible]

消息订阅

```
// 订阅主题
int MQTTAsync_subscribe(MQTTAsync handle, const char* topic, int qos,
                        MQTTAsync_responseOptions* response);

// 取消订阅
int MQTTAsync_unsubscribe(MQTTAsync handle, const char* topic,
                           MQTTAsync_responseOptions* response);

// 设置回调
int MQTTAsync_setCallbacks(MQTTAsync handle, void* context,
                           MQTTAsync_connectionLost* cl,
                           MQTTAsync_messageArrived* ma,
                           MQTTAsync_deliveryComplete* dc);
```

核心数据结构

连接选项

```
typedef struct {
    int keepAliveInterval;           // 保持活跃间隔（秒）
    int cleansession;                // 清除会话标志
    int maxInflight;                 // 最大飞行消息数
    MQTTAsync_willOptions* will;     // 遗嘱消息
    MQTTAsync_SSLOptions* ssl;       // SSL/TLS 选项
    MQTTAsync_onSuccess* onSuccess;  // 成功回调
    MQTTAsync_onFailure* onFailure;  // 失败回调
    void* context;                   // 用户上下文
    int MQTTVersion;                 // MQTT 版本
} MQTTAsync_connectOptions;
```

SSL 选项

```
typedef struct {
    const char* trustStore;          // CA 证书路径
    const char* keyStore;             // 客户端证书路径
    const char* privateKey;           // 私钥路径
    const char* privateKeyPassword;   // 私钥密码
    const char* enabledCiphersuites;  // 启用的加密套件
    int enableServerCertAuth;         // 启用服务器证书验证
} MQTTAsync_SSLOptions;
```

六、使用方法

1. 基本连接示例

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "MQTTAsync.h"

#define BROKER_ADDRESS    "tcp://broker.emqx.io:1883"
#define CLIENT_ID        "OH_HI3519DV500_Client"
#define TOPIC            "test/topic"
#define PAYLOAD          "Hello from hi3519dv500"
#define QOS              1
#define TIMEOUT          10000L

volatile int finished = 0;

void onConnect(void* context, MQTTAsync_successData* response) {
    printf("Connected successfully\n");
    finished = 1;
}

void onConnectFailure(void* context, MQTTAsync_failureData* response) {
    printf("Connect failed, rc %d\n", response ? response->code : 0);
    finished = 1;
}

int main(int argc, char* argv[]) {
    MQTTAsync client;
    MQTTAsync_connectOptions conn_opts = MQTTAsync_connectOptions_initializer;
    int rc;

    rc = MQTTAsync_create(&client, BROKER_ADDRESS, CLIENT_ID,
                        MQTTCLIENT_PERSISTENCE_NONE, NULL);
    if (rc != MQTTASYNC_SUCCESS) {
        printf("Failed to create client, return code %d\n", rc);
        return rc;
    }

    conn_opts.keepAliveInterval = 20;
    conn_opts.cleansession = 1;
    conn_opts.onSuccess = onConnect;
    conn_opts.onFailure = onConnectFailure;
    conn_opts.context = client;

    rc = MQTTAsync_connect(client, &conn_opts);
    if (rc != MQTTASYNC_SUCCESS) {
        printf("Failed to start connect, return code %d\n", rc);
        return rc;
    }
}
```

```

while (!finished) {
    usleep(10000);
}

MQTTAsync_destroy(&client);
return 0;
}

```

2. 编译应用程序

方法 1: 使用 OpenHarmony GN 构建系统 (推荐)

创建 BUILD.gn 文件:

```

import("//build/ohos.gni")

ohos_executable("mqtt_test") {
    sources = [ "mqtt_test.c" ]

    include_dirs = [
        "//third_party/ph_mqtt/src"
    ]

    deps = [
        "//third_party/ph_mqtt:libpaho-mqtt3as"
    ]

    part_name = "mqtt_test"
    subsystem_name = "applications"
}

```

编译:

```

python3 build.py --product-name ipcamera_hispark_hi3519dv500_linux \
    --build-target mqtt_test --ccache

```

方法 2: 交叉编译 (使用 OpenHarmony 工具链)

```

aarch64-v01c01-linux-musl-gcc \
    --sysroot=out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/sysroot \
    -o mqtt_test mqtt_test.c \
    -Ithird_party/ph_mqtt/src \
    -Lout/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux \
    -lpaho-mqtt3as -lssl_openssl -lcrypto_openssl

```

3. 部署到设备

```
# 推送库文件到设备
hdc file send libpaho-mqtt3as.so /system/lib/
hdc file send libssl_openssl.so /system/lib/
hdc file send libcrypto_openssl.so /system/lib/

# 推送测试程序
hdc file send mqtt_test /data/
hdc shell chmod +x /data/mqtt_test

# 运行测试
hdc shell "cd /data && LD_LIBRARY_PATH=/system/lib ./mqtt_test"
```

七、与其他组件的关系

依赖关系

组件	说明	状态
openssl	SSL/TLS 加密，MQTT SSL 连接必需	已整合
musl (sysroot_lite)	C 标准库（lite 系统构建依赖）	系统内置

配合使用

ph_mqtt 可以与已整合的其他组件配合使用：

1. 与 BusyBox 配合进行网络调试：

```
busybox ping -c 4 broker.emqx.io
busybox telnet broker.emqx.io 1883
```

2. 与 iptables 配合管理 MQTT 端口：

```
iptables -A INPUT -p tcp --dport 1883 -j ACCEPT
iptables -A INPUT -p tcp --dport 8883 -j ACCEPT
```

3. 与 tcpdump 配合抓取 MQTT 报文：

```
tcpdump -i eth0 -nn port 1883
```

八、注意事项

1. 架构差异

本产品 (hispark_hi3519dv500_linux) 使用 ARM64 架构, 与 hispark_taurus_linux (ARM 32-bit) 存在差异:

维度	hi3519dv500_linux	hispark_taurus_linux
架构	ARM64 (aarch64)	ARM32 (armv7)
编译工具链	aarch64-v01c01-linux-musl-gcc	arm-linux-musleabi-gcc
系统类型	small	small
musl sockaddr_storage	被 __MUSL__ 守卫跳过	直接由 musl 提供
产物格式	ELF 64-bit LSB	ELF 32-bit LSB

2. -D_UAPI_LINUX_SOCKET_H 是 ARM64/musl 通用问题

该宏在 hi3519dv500 (ARM64/musl) 上会导致 sockaddr_storage 未定义, 与 BusyBox、iptables、libpcap 遇到的问题完全相同。任何从 taurus (ARM32) 移植过来的组件如果 BUILD.gn 含此宏, 都需要移除。如果后续 ph_mqtt 需要移植到其他 ARM64 + musl 的平台 (如 hispark_hi3516dv500_linux), 同样需要此修改。

3. OpenSSL 依赖

ph_mqtt 编译依赖 OpenSSL, 确保目标平台的 config.json 中已包含 openssl 组件。OpenSSL 需要先于 ph_mqtt 编译完成 (构建系统自动处理依赖顺序)。

4. 产物安装路径

编译产物通过 fs.yml 中 libs → /lib 映射自动打包进 rootfs。全量编译后, libpaho-mqtt3as.so 会出现在设备 /lib/ 目录下。

5. SSL/TLS 证书

使用 SSL/TLS 连接时, 需将 CA 证书推送到设备:

```
hdc file send ca.crt /data/
```

6. 线程安全

MQTTAsync API 是线程安全的, 可以在多线程环境中使用。库内部使用 pthread 进行线程管理。

九、参考资源

- Eclipse Paho 官方文档: <https://www.eclipse.org/paho/>
- MQTT 3.1.1 规范: <http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>
- MQTT 5.0 规范: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- OpenHarmony 文档: <https://gitee.com/openharmony>